LIGA ROBÓTICA NEUQUINA · GUÍA PARA INFRAESTRUCTURA

Despliegue del sistema en nube propia con Docker

Qué es el sistema, qué necesita de infraestructura, cómo construir y correr la imagen, y qué opciones hay para la base de datos, incluida la migración a servidores propios. La versión en el repositorio es `docs/DEPLOY.md`.

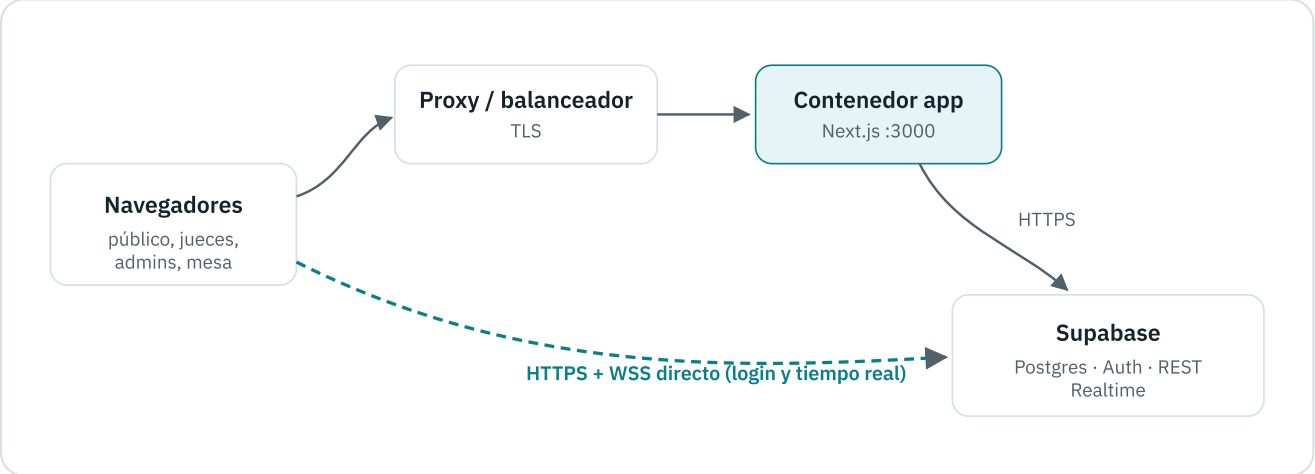
Build standalone y health check verificados

Sin recursos externos en runtime

Imagen Docker sin construir todavía (no hubo Docker en la máquina de desarrollo)

01 Cómo se conecta

El sistema tiene dos piezas. **La aplicación** es Next.js 16 sobre Node.js 24: un solo proceso HTTP en el puerto `3000`, sin estado (la sesión viaja en cookies, no escribe archivos). Es lo que va en el contenedor. **Supabase** es PostgreSQL más tres servicios que la app usa directamente: Auth (login de administradores), la API REST (todas las consultas) y Realtime (marcadores y relojes en vivo por WebSocket).



La flecha punteada es la que más se olvida.
Los navegadores se conectan directo a Supabase, no a través de la app. La URL de Supabase tiene que ser alcanzable desde donde estén los usuarios: internet para el público, y el wifi del evento para las tablets de los jueces y la mesa de acreditación.

02 Qué se necesita

Para la aplicación

- [ ] Construir imágenes Docker (runner de CI con Docker, BuildKit o Kaniko) y un registro para publicarlas.
- [ ] Acceso a un registry npm durante el build: el público o un espejo interno (Nexus, Artifactory). En runtime no hace falta.
- [ ] Un contenedor alcanza para una jornada completa. Sugerido: 1 GB de RAM de límite (uso estimado 200–300 MB) y 0,5 a 1 vCPU.
- [ ] Dominio y certificado TLS en el proxy. El sitio público tiene que ser accesible desde internet: lo miran familias y equipos desde el celular.
- [ ] Salida HTTPS desde el contenedor hacia la URL de Supabase.
- [ ] Gestión de secretos para `SUPABASE_SERVICE_ROLE_KEY` (secret de Kubernetes, Vault o variable protegida). Nunca va en la imagen.
- [ ] Logs: la app escribe a stdout y stderr.
- [ ] Health checks: `GET /api/health` (liveness) y `GET /api/health?deep=1` (readiness, verifica Supabase).

Para la base de datos

Depende de la opción elegida en la sección Base de datos.

03 Variables

Next.js incrusta las variables `NEXT_PUBLIC_*` en el JavaScript del navegador durante el build, así que hay variables de `build` y de `runtime`. En la práctica: **una imagen por ambiente**. Test y producción apuntan a Supabase distintos, entonces son imágenes distintas. Plantilla en el repo: `.env.docker.example`.

VARIABLE	CUÁNDO	OBLIGATORIA	PARA QUÉ
`NEXT_PUBLIC_SUPABASE_URL`	`build`	Sí	URL de Supabase tal como la ven los navegadores.
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	`build`	Sí	Clave pública de Supabase. Es pública por diseño: la protege la seguridad por filas de la base.
`SITE_URL`	`build` `runtime`	Recomendada	URL pública del sistema, sin barra final. Se usa para las vistas previas al compartir links.

VARIABLE	CUÁNDO	OBLIGATORIA	PARA QUÉ
`SUPABASE_SERVICE_ROLE_KEY`	`runtime` `secreta`	Sí	Clave de servicio que saltea la seguridad de la base. Solo la usa el servidor: juez, acreditación, inscripción y alta de usuarios.
`SERVER_ACTIONS_ALLOWED_ORIGINS`	`build`	Si hace falta	Cuando el proxy reescribe el Host. Ver Proxy.
`NEXT_SERVER_ACTIONS_ENCRYPTION_KEY`	`build` `secreta`	Con más de una réplica	Misma clave en todas las réplicas. Ver Réplicas.
`DEPLOYMENT_ID`	`build`	Con más de una réplica	Identifica la versión durante un deploy gradual.
`NPM_CONFIG_REGISTRY`	`build`	Sin internet	Espejo npm interno.
`PORT` / `HOSTNAME`	`runtime`	No	Por defecto `3000` y `0.0.0.0`.

04 Construir y correr

La imagen es multi-etapa sobre `node:24-alpine`. La etapa final solo lleva el build standalone (unos 40 MB de aplicación más Node), corre como usuario sin privilegios (`nextjs`, uid 1001), admite sistema de archivos de solo lectura y trae su propio `HEALTHCHECK`. El build falla con un mensaje claro si faltan las dos variables obligatorias.

```
docker build \
  --build-arg NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co \
  --build-arg NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ... \
  --build-arg SITE_URL=https://liga.ejemplo.gob.ar \
  -t registry.interno/sistema-lrn:1.0.0 .

docker run -d --name sistema-lrn -p 3000:3000 \
  -e SUPABASE_SERVICE_ROLE_KEY=eyJ... \
  -e SITE_URL=https://liga.ejemplo.gob.ar \
  --read-only --tmpfs /tmp --tmpfs /app/.next/cache \
  registry.interno/sistema-lrn:1.0.0

curl http://localhost:3000/api/health?deep=1
# {"status":"ok","supabase":"ok","ms":42}
```

En el repo también hay un `docker-compose.yml` de ejemplo (una réplica, solo lectura, sin privilegios, rotación de logs):

```
cp .env.docker.example .env.docker   # completar
docker compose --env-file .env.docker up -d --build
```

Kubernetes (ejemplo)

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: sistema-lrn }
spec:
  replicas: 1
  selector: { matchLabels: { app: sistema-lrn } }
  template:
    metadata: { labels: { app: sistema-lrn } }
    spec:
      securityContext: { runAsNonRoot: true, runAsUser: 1001 }
      containers:
        - name: app
          image: registry.interno/sistema-lrn:1.0.0
          ports: [{ containerPort: 3000 }]
          env:
            - { name: SITE_URL, value: "https://liga.ejemplo.gob.ar" }
            - { name: TZ, value: "America/Argentina/Buenos_Aires" }
          envFrom: [{ secretRef: { name: sistema-lrn } }]   #
SUPABASE_SERVICE_ROLE_KEY
          resources:
            requests: { cpu: 250m, memory: 256Mi }
            limits:   { cpu: "1",  memory: 1Gi }
          livenessProbe:  { httpGet: { path: /api/health, port: 3000 },
periodSeconds: 30 }
          readinessProbe: { httpGet: { path: "/api/health?deep=1", port: 3000
}, periodSeconds: 15 }
          securityContext: { readOnlyRootFilesystem: true,
allowPrivilegeEscalation: false }
          volumeMounts:
            - { name: tmp, mountPath: /tmp }
            - { name: next-cache, mountPath: /app/.next/cache }
      volumes:
        - { name: tmp, emptyDir: {} }
        - { name: next-cache, emptyDir: {} }
```

> **Por qué la readiness usa `?deep=1` y la liveness no.**
> Si Supabase no responde, conviene sacar la réplica del balanceador, pero reiniciarla no arregla nada.

05 Proxy y balanceador

- TLS termina en el proxy. La app habla HTTP plano en `:3000`.

- **Reenviar el `Host` original** (o `X-Forwarded-Host`) y `X-Forwarded-Proto: https`. Los botones del panel comparan el origen del navegador con el Host como protección contra CSRF. Si el proxy lo reescribe, fallan con *Invalid Server Actions request*. Si no se puede preservar, construir la imagen con `SERVER_ACTIONS_ALLOWED_ORIGINS=liga.ejemplo.gob.ar`.
- Timeouts de al menos 60 segundos: la exportación a Excel de un evento grande tarda unos segundos.
- No hacen falta WebSockets hacia la app: el tiempo real va del navegador directo a Supabase.
- `/_next/static/*` es inmutable (lleva un hash en el nombre) y se puede cachear para siempre. No cachear el HTML ni `/api/*`: todas las páginas son dinámicas.

06 Réplicas y actualizaciones

Una réplica es suficiente y es lo más simple. Con más de una:

- Definir `NEXT_SERVER_ACTIONS_ENCRYPTION_KEY` en el build, la misma en todas las réplicas. Se genera una vez con `openssl rand -base64 32`. Sin esto, los botones fallan al azar con *Failed to find Server Action*.
- Definir `DEPLOYMENT_ID` (por ejemplo, el tag de la imagen): durante un deploy gradual, un navegador con la versión vieja recarga en vez de romperse.
- No hacen falta sesiones pegajosas ni caché compartido (Redis).

Para actualizar: construir un tag nuevo y hacer rolling update. Si la versión trae archivos nuevos en `supabase/migrations/`, aplicar esa migración a la base **antes** de desplegar la imagen. Cada migración se aplica una sola vez y en orden, así que conviene llevar registro de la última aplicada en cada ambiente.

07 Base de datos

La app depende de Supabase completo, no solo de PostgreSQL: usa su login, su API REST y su tiempo real, y la seguridad está escrita como reglas por fila que usan funciones y roles de Supabase (`auth.uid()`, `anon`, `authenticated`, `service_role`).

RECOMENDADA PARA EMPEZAR

OPCIÓN A

Seguir en Supabase Cloud

Esfuerzo	Ninguno
Datos	Afuera (São Paulo)
Cambios	Ninguno

Solo se mueve la app. Necesita salida HTTPS hacia Supabase y que los navegadores lleguen por HTTPS y WSS. Conviene el plan Pro: el gratis pausa el proyecto tras 7 días sin uso y no tiene backups diarios.

VIABLE

OPCIÓN B

Supabase propio

Esfuerzo	1 a 2 días de infra
Datos	Adentro
Cambios	Solo variables y rebuild

Se instala el stack oficial de Supabase con Docker y se mudan los datos. Mínimo del stack: 4 GB de RAM, 2 núcleos y 40 GB de SSD, más backups propios.

REQUIERE REESCRIBIR

OPCIÓN C

Solo PostgreSQL propio

Esfuerzo	Proyecto aparte
Datos	Adentro
Cambios	Login, consultas, tiempo real

Habría que reemplazar el login, todas las consultas y el tiempo real, y adaptar la seguridad de la base. Tampoco alcanza con apuntar Supabase a un Postgres genérico: necesita su propia imagen con extensiones y roles.

Qué pide la opción B (Supabase propio)

- Stack oficial: supabase.com/docs/guides/self-hosting/docker (versión de la documentación: `self-hosted/v0.8.2`). Servicios: Postgres, Auth, PostgREST, Realtime, gateway Envoy en el puerto `8000`, Studio, postgres-meta, Supavisor, Storage y Edge Runtime. Logflare, Vector e imgproxy son opcionales. El sistema no usa Storage ni Edge Functions.
- Un dominio con TLS para la API (por ejemplo `api-liga.ejemplo.gob.ar` hacia el gateway), alcanzable desde internet y con WebSocket habilitado. Esa URL es la `NEXT_PUBLIC_SUPABASE_URL` de la app.
- Secretos que se generan con `utils/generate-keys.sh` y `utils/add-new-auth-keys.sh`: `POSTGRES_PASSWORD`, `JWT_SECRET`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `DASHBOARD_USERNAME`, `DASHBOARD_PASSWORD`, `SECRET_KEY_BASE`, `REALTIME_DB_ENC_KEY`, `VAULT_ENC_KEY` y `PG_META_CRYPTO_KEY`.
- URLs: `SUPABASE_PUBLIC_URL` y `API_EXTERNAL_URL` apuntan al dominio de la API; `SITE_URL`, al de la app.
- Desactivar el registro público de usuarios en Auth, porque los usuarios los crea el superusuario desde el panel. En versiones actuales la variable es `DISABLE_SIGNUP`; verificar el nombre en la versión instalada. No hace falta configurar correo: el sistema no manda mails.
- El panel Studio no debe quedar expuesto a internet.

08 Migrar la base (opción B)

1. **Levantar el Supabase propio vacío** y comprobar que responde en `/rest/v1/` con la anon key.
2. **Crear la estructura** ejecutando `supabase/schema.sql` en Studio o con `psql`. Crea tablas, funciones, reglas de seguridad y la publicación de tiempo real, y siembra disciplinas y categorías.
3. **Congelar cambios** en el sistema actual, fuera de una jornada.
4. **Exportar los datos** del Cloud. El string de conexión está en Project Settings, sección Database.

```
# Usuarios de login (las contraseñas viajan ya cifradas)
pg_dump "$CLOUD_DB_URL" --data-only \
  --table=auth.users --table=auth.identities > auth.sql

# Datos del sistema
pg_dump "$CLOUD_DB_URL" --data-only --schema=public > public.sql
```

5. **Importar** con el usuario administrador de Postgres (en self-hosted, `supabase_admin`) y los triggers desactivados durante la carga, para que no reescriban fechas ni dupliquen filas automáticas:

```
# disciplines y categories ya las sembró schema.sql
psql "$NEW_DB_URL" -c "truncate disciplines, categories cascade;"

PGOPTIONS='-c session_replication_role=replica' \
  psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f auth.sql
PGOPTIONS='-c session_replication_role=replica' \
  psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f public.sql
```

   Si `auth.sql` falla por columnas que no coinciden, el Auth del Cloud y el propio son de versiones distintas. Volver a exportar con `--column-inserts` y quitar las columnas que el destino no tenga. Como son pocos usuarios (solo administradores), también se pueden recrear desde la pantalla Usuarios del panel.

6. **Reconstruir la imagen** con la nueva URL y anon key, y desplegarla con la nueva service role key.
7. **Verificar** con la lista de la sección Verificación.

QUÉ PASA CON	RESULTADO
Contraseñas de los administradores	Se conservan.
Sesiones abiertas	Se pierden: cada administrador inicia sesión de nuevo, una vez.
Links de juez y de acreditación	Siguen funcionando. Si cambia el dominio de la app, hay que volver a compartirlos.

09 Backups

Supabase Cloud Pro hace backups diarios automáticos. En self-hosted son responsabilidad propia; como mínimo, un volcado diario guardado fuera del servidor:

```
pg_dump "$DB_URL" --format=custom --schema=public --schema=auth \
  -f "lrn-$(date +%F).dump"
```

Probar una restauración al menos una vez antes de la primera jornada.

10 Verificación después de desplegar

1. `GET /api/health?deep=1` responde `{"status":"ok","supabase":"ok"}`.
2. Iniciar sesión en `/admin` y abrir un evento.
3. Tocar un botón que guarde algo, por ejemplo abrir y cerrar una inscripción. Si falla con *Invalid Server Actions request*, ver Proxy.
4. Abrir el link de un juez en una tablet y la pantalla del evento en otra. Iniciar un partido desde el juez y ver que el reloj arranca solo en la pantalla: eso confirma que el tiempo real llega a Supabase.
5. Exportar el Excel de un evento.
6. Abrir el sitio desde un celular con datos móviles, para confirmar el acceso desde internet.

11 Problemas comunes

SÍNTOMA	CAUSA PROBABLE	SOLUCIÓN
El build falla en `npm ci`	Sin acceso al registry npm	`NPM_CONFIG_REGISTRY` con el espejo interno
El build falla: faltan `--build-arg`	No se pasaron las `NEXT_PUBLIC_*`	Pasarlas como argumentos del build
Botones: *Invalid Server Actions request*	El proxy reescribe el Host	Preservar el Host o usar `SERVER_ACTIONS_ALLOWED_ORIGINS`
Botones que fallan al azar: *Failed to find Server Action*	Réplicas con builds distintos	La misma imagen en todas y `NEXT_SERVER_ACTIONS_ENCRYPTION_KEY`
Relojes y marcadores no se actualizan solos	Los navegadores no llegan a Supabase por WSS	Habilitar WebSocket hacia la URL de Supabase
No se puede iniciar sesión	El navegador no llega a Supabase Auth	La URL de Supabase tiene que ser alcanzable desde el navegador
`/api/health?deep=1` devuelve 503	El contenedor no llega a Supabase	Revisar salida HTTPS, DNS o proxy de salida
Juez o acreditación: error 500	Falta `SUPABASE_SERVICE_ROLE_KEY` en runtime	Cargar el secreto en el contenedor

Sistema de la Liga Robótica Neuquina · Guía de despliegue, septiembre de 2026 · Fuente en el repositorio: `docs/DEPLOY.md`, `Dockerfile`, `docker-compose.yml`, `.env.docker.example`.